IT305 COMPUTER NETWORKS LAB

# Fault-Tolerant Topic-Based File Distribution Service

Final implementation and result analysis
Groups 099 and 107
Submission deadline: 16 October 2026

| Member | Roll number | Individual contribution |
|---|---|---|
| Jenish Gandhi | 202401426 | Not supplied by member |
| Chandan Luhar | 202401431 | Not supplied by member |
| Nirav Rana | 202401456 | Not supplied by member |
| Shubh Patel | 202401478 | Not supplied by member |
| Yashraj Solanki | 202401481 | Not supplied by member |
| Yash Oza | 202401493 | Not supplied by member |

## Completion status

Enter the actual individual contributions in team_members.csv.

Run the required physical three-PC Animals10 experiment and regenerate this report.

Language: C11 with POSIX TCP sockets and pthreads. Python is used for tests, measurements, graphs and report generation.

## 1. Architecture and topic mapping

The server resolves the CLI dataset root to an absolute path and creates a dynamic array of Topic records. Each entry stores the topic key, absolute directory path and immutable file manifest. Linear lookup is sufficient for ten topics. Each manifest stores sorted regular filenames, 64-bit sizes, total bytes and an FNV-1a metadata fingerprint.

To follow the key configuration approach, an optional topics.conf in the dataset root maps topic keys to relative directories using a TAB separator. Paths are validated and converted to absolute paths. If no configuration exists, immediate topic directories are discovered automatically. Thus the exact required CLI works without any extra required argument.

Each Part folder is self-contained: server.c, client.c, common.h, config.h and Makefile. CASE_MODE selects sequential/concurrent, restart, resume or parallel resume. No Shared folder, fixed server IP, fixed dataset path or required nonstandard argument is used.

## 2. Wire protocol and correspondence to the key

| Key role | Implemented format / meaning |
|---|---|
| GET | GET <topic> + LF; one request per TCP connection. |
| OK + file list | OK <count> <fingerprint> <total_bytes> + LF. Then, per file: u64 filename length, exact filename bytes, u64 size. |
| ERR | ERR <reason> + LF. Unknown topics, changed datasets and invalid checkpoints are rejected. |
| START | START <session_id> <lane> <lanes> <file_index> <offset> <fingerprint> + LF. READY confirms validation. |
| FILE_HDR | FILE <file_index> <start_offset> + LF. Name and full size are already known from the manifest. |
| DATA | Unsigned big-endian u64 block length, followed by exactly that many binary bytes; maximum 65,536 bytes. |
| ACK | Unsigned big-endian u64 next offset, sent after a complete successful file write. |
| File end | A zero block length ends a file; the client acknowledges its final offset. |
| CLOSE | DONE + LF, followed by client BYE + LF. |

Control messages are newline-framed; binary fields and payloads are length-framed. send_all/recv_all handle partial operations and EINTR. Only one request and one unacknowledged data block exist per connection, so connection state, file index and expected offset associate responses with the transaction. Session/lane identifiers are negotiated in START instead of repeated in every frame. This is a documented compact protocol alternative to the sample key.

## 3. Part I: sequential and multithreaded service

In single mode, the accept loop calls serve directly; the next client is handled only after the current transfer ends. In multi mode, the accept loop creates one detached worker per connection. Workers share read-only topic manifests. Each client creates output_dir/topic and receives every regular file. The server defaults to multi when the optional mode is omitted.

## 4. Part II Case 1: restart without a progress session

An interrupted client discards its received topic files and in-memory manifest/checkpoint, reconnects, and sends START with file index 0 and offset 0. The server retains no resumable state in this mode. A random identifier remains only for log correlation. Completed files and blocks from the failed attempt are transmitted again.

## 5. Part II Case 2: checkpoint and session management

The client retains its topic manifest, 64-bit session ID, file index and next byte offset in RAM. The server retains a mutex-protected linked list of session/lane state, including the fingerprint and last acknowledged checkpoint. The client receives the whole block into RAM, writes it completely, advances the checkpoint and sends its offset ACK. An interrupted partial block is discarded.

After reconnection, the manifest is checked and START presents the saved session/lane checkpoint. The server validates topic fingerprint, lane assignment, file index and offset bounds, seeks to that position and resumes. A client checkpoint may be ahead of the server record when an ACK is lost; the client checkpoint is accepted after validation. Trusted lab clients are assumed. No power-loss or client-process-crash durability is claimed.

## 6. Fault injection

The CLI failure probability is applied independently before every non-empty payload block of at most 64 KiB. With probability p, the server sends half the block and closes the connection. This tests an actual incomplete message. Otherwise the complete block is sent and acknowledged. Per-connection rand_r state avoids a shared random generator. At p=0 no fault is injected; at p=1 a non-empty transfer cannot complete. The default limit is 1000 attempts per lane, configurable through FT_MAX_ATTEMPTS, with a 10 ms retry delay.

## 7. Part II Case 2 Enhanced: persistent parallel lanes

Two client threads and TCP connections are used by default; an optional final argument selects 1-16. Sorted file i is assigned to lane i modulo K. A lane keeps its connection open for all its assigned files and has its own checkpoint and server worker. Only an interrupted lane reconnects. Each file has one writer, so merging and file-write locks are unnecessary. Initial connection and manifest reception are covered by the retry loop.

Whole-file partitioning was chosen over byte-range splitting to keep recovery and output ownership simple. Blocking sockets were chosen over epoll for this small client population. Persistent lanes avoid the previous per-file connection and repeated whole-manifest overhead. Large unequal files may leave one lane busy after another finishes; more lanes do not guarantee a speedup.

## 8. Design changes, correctness and limits

The fixed file-to-lane assignment and two-lane default now agree with the submitted design. The later experimental dynamic queue/adaptive lane heuristic was removed. The original submitted design is preserved in Documents. The final source folders are independent; obsolete Shared-folder instructions and historical graphs were removed.

Dataset metadata is checked against the startup snapshot before serving and before successful completion, and each opened file is checked again. A changed directory, file size or modification time rejects an old session. This is not a cryptographic content signature: the dataset must remain immutable during an experiment. SHA-256 verification is performed separately by the test and LAN tools.

Topic/file validation rejects traversal and file separators. Symlink files are skipped by server discovery and rejected on output. Socket operations default to 300-second inactivity timeouts, configurable by FT_IO_TIMEOUT (1-3600 seconds) and use MSG_NOSIGNAL. CSV measurements go to stdout; progress and faults go to stderr. Diagnostic pipes in tests are drained, and benchmark logs use regular files. Session records are limited to 4096 lanes and retained until server exit; the runner restarts the server for each batch.

## 9. Validation performed

| Check | Observed result |
|---|---|
| strict C builds | 4 folders passed -Wall -Wextra -Wpedantic -Werror |
| functional checks | 39 passed on current revision |
| stream disconnect checks | 24 passed on current revision |
| targeted regressions | 6 passed on current revision |
| orchestration rehearsal | 20 configurations passed on current revision; one PC only |
| fresh local benchmark | 60 successful SHA-256-verified synthetic loopback batches on current revision |
| wifi revision checks | 32-second queue, two-port diagnosis and proxy-bypass checks passed |
| Windows PowerShell | Script reviewed; not runtime-tested in this Linux environment |
| physical three pc tests | Not performed here; run on the user PCs |
| historical evidence | Uploaded original real-image logs retained; not rerun here |

The current revision reruns functional, interrupted-stream and regression checks; current logs are saved separately from the supplied historical logs. A local test queues a Part I client for 32 seconds to verify that slow sequential service no longer hits the old 30-second wait limit. PowerShell and physical hotspot operation cannot be executed in this Linux environment.

Real-image tests use a small sample from all ten Animals10 topics, including a 5,574,493-byte cat image. They verify binary transfer and byte-offset recovery, and are not full-dataset LAN measurements. The orchestration rehearsal runs A/B/C as processes on one machine and is explicitly labelled as a rehearsal.

## 10. Dataset and experiment method

Animals10 folders are the topic keys. The complete version-2 archive inventory contains 26,179 images plus translate.py outside raw-img; only regular files in topic directories are served. Images are treated as opaque binary files. Every cane image is smaller than 64 KiB, while gatto includes larger files suitable for demonstrating nonzero offset resume.

| Topic | Images | Payload MiB | 64 KiB blocks |
|---|---|---|---|
| cane | 4863 | 57.88 | 4863 |

| Topic | Images | Payload MiB | 64 KiB blocks |
|---|---|---|---|
| cavallo | 2623 | 35.69 | 2623 |
| elefante | 1446 | 55.8 | 1922 |
| farfalla | 2112 | 59.01 | 2497 |
| gallina | 3098 | 49.44 | 3098 |
| gatto | 1668 | 144.14 | 3563 |
| mucca | 1866 | 24.64 | 1866 |
| pecora | 1820 | 59.73 | 2262 |
| ragno | 4821 | 75.34 | 4987 |
| scoiattolo | 1862 | 24.23 | 1862 |

Required topology: PC A serves the unchanged dataset; PCs B and C each run client processes. Part I uses 1, 2, 4 and 8 total clients in each mode. Part II uses 4 clients and four failure probabilities, including zero. Each setting is repeated three times: 60 batches in total. The one-client baseline uses B only. Enhanced uses two TCP lanes per client unless explicitly recorded otherwise.

Let B be the sum of ceil(file_size/65536) for non-empty files, and K=max(10,B). The runner uses p={0,0.1/K,0.5/K,1/K}. This documented change from the original design schedule avoids an effectively impossible full-topic restart at high per-block probabilities on thousands of files. For cane, B=4863. The exact probabilities are saved in environment.json.

Wi-Fi/WSL revision: both transfer and coordination ports are checked before registration. NAT forwarding and narrowly scoped firewall rules are supplied in Tools/windows/Setup-WSL-Network.ps1. Tools/network_check.py separates listener failures from forwarding and remote reachability problems. Control HTTP bypasses environment proxies and retries transient errors. These changes do not remove hotspot client isolation; use a network that permits peer traffic.

A coordination-only Python HTTP service on port 9101 schedules the unchanged C binaries on transfer port 9000. Node agents estimate clock offsets using the lowest round-trip clock probe and record the uncertainty. NTP-synchronised clocks and a stable LAN are preferred. Hash checking and report generation happen outside each timed transfer. The server and both client-node CSVs, progress logs, source hashes, platform details and run schedule are preserved.

## 11. Metrics

U = total bytes in successfully completed, SHA-256-verified client outputs. S = all server payload bytes accepted by send, including partial failed blocks and retransmitted data. For a fully successful batch, R = S - U. T = latest client finish minus earliest client start, including reconnect delays and queue waits. Goodput = U/(1048576*T) MiB/s; payload rate = S/(1048576*T). Redundancy = 100*R/U percent. Protocol bytes = server application bytes sent and received minus S.

Do not sum overlapping connection durations or average client rates. Failed batches remain in summary.csv with status FAILED; successful-transfer redundancy is left undefined for those batches. These counters do not include TCP/IP headers or kernel-level TCP retransmissions.

## 12. Physical three-PC Animals10 results

PENDING: no physical three-PC measurements have been provided. Run Tools/lan.py on PCs A, B and C using THREE_PC_TEST_GUIDE.md, then run Tools/finalize.py --lan Results/lan. This section will be regenerated from the measured CSVs and figures. No LAN speedup is claimed.

## 13. Local synthetic validation results

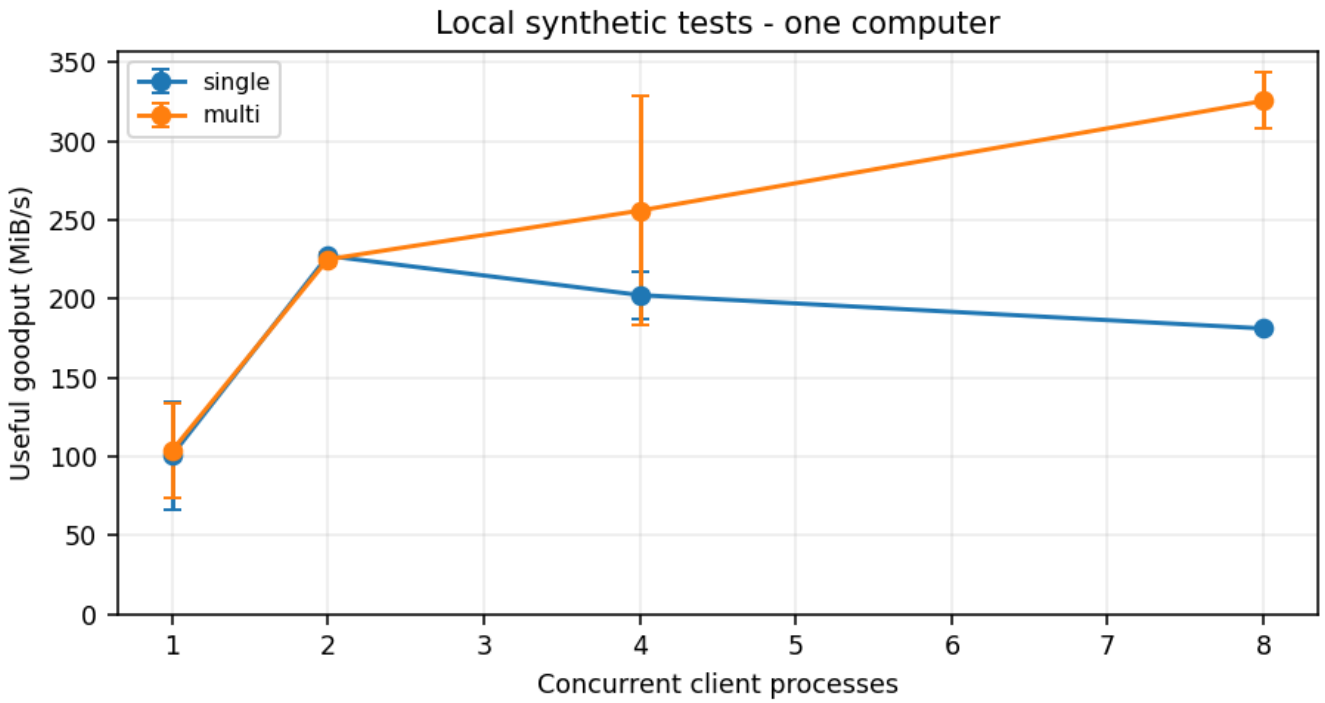
60 of 60 batches completed with all client hashes verified. Failed runs remain in the raw CSVs and are excluded from successful-transfer averages.

Environment: IPv4 loopback, one computer. Topic: cane.

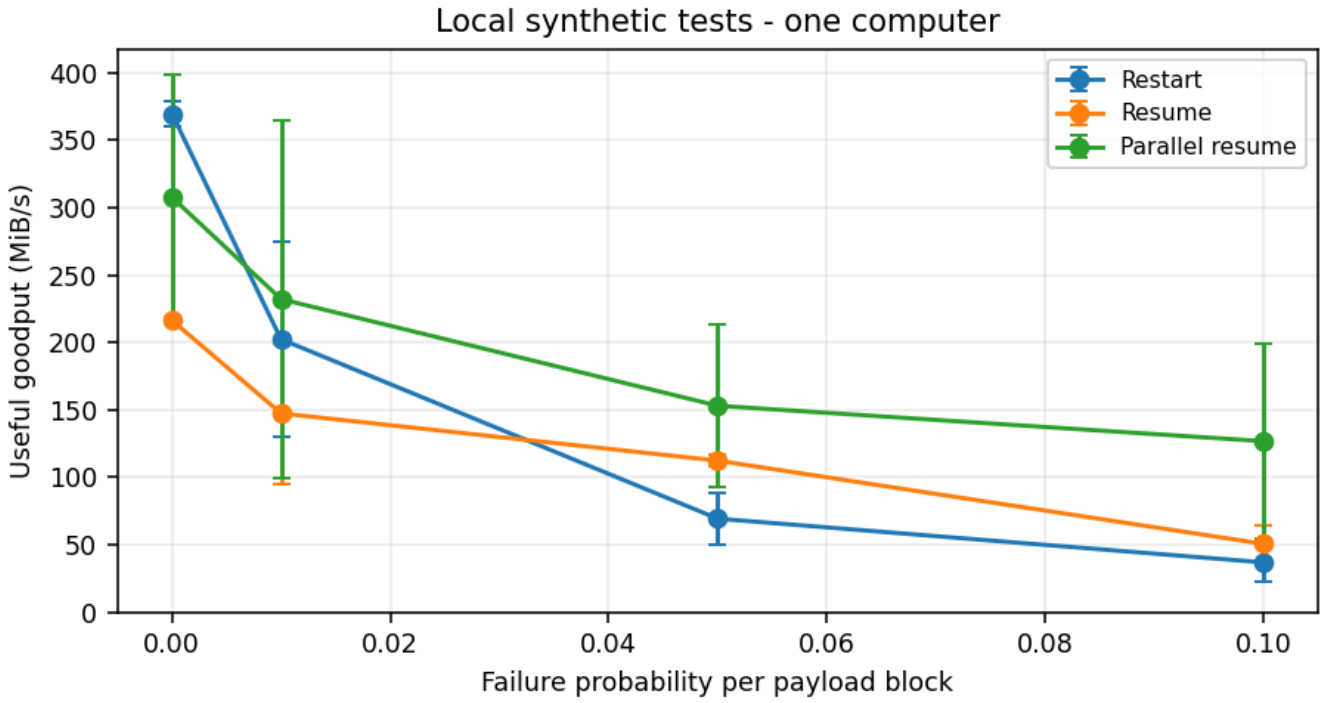
Linux-6.18.55-x86_64-with-glibc2.39; gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0

| Part I mode | Clients | Runs | Goodput mean +/- SD (MiB/s) |
|---|---|---|---|
| multi | 1 | 3 | 103.64 +/- 30.28 |
| multi | 2 | 3 | 224.96 +/- 4.25 |
| multi | 4 | 3 | 255.84 +/- 72.87 |
| multi | 8 | 3 | 325.40 +/- 17.87 |
| single | 1 | 3 | 100.62 +/- 34.29 |
| single | 2 | 3 | 227.14 +/- 1.60 |
| single | 4 | 3 | 202.32 +/- 15.21 |
| single | 8 | 3 | 181.12 +/- 1.75 |

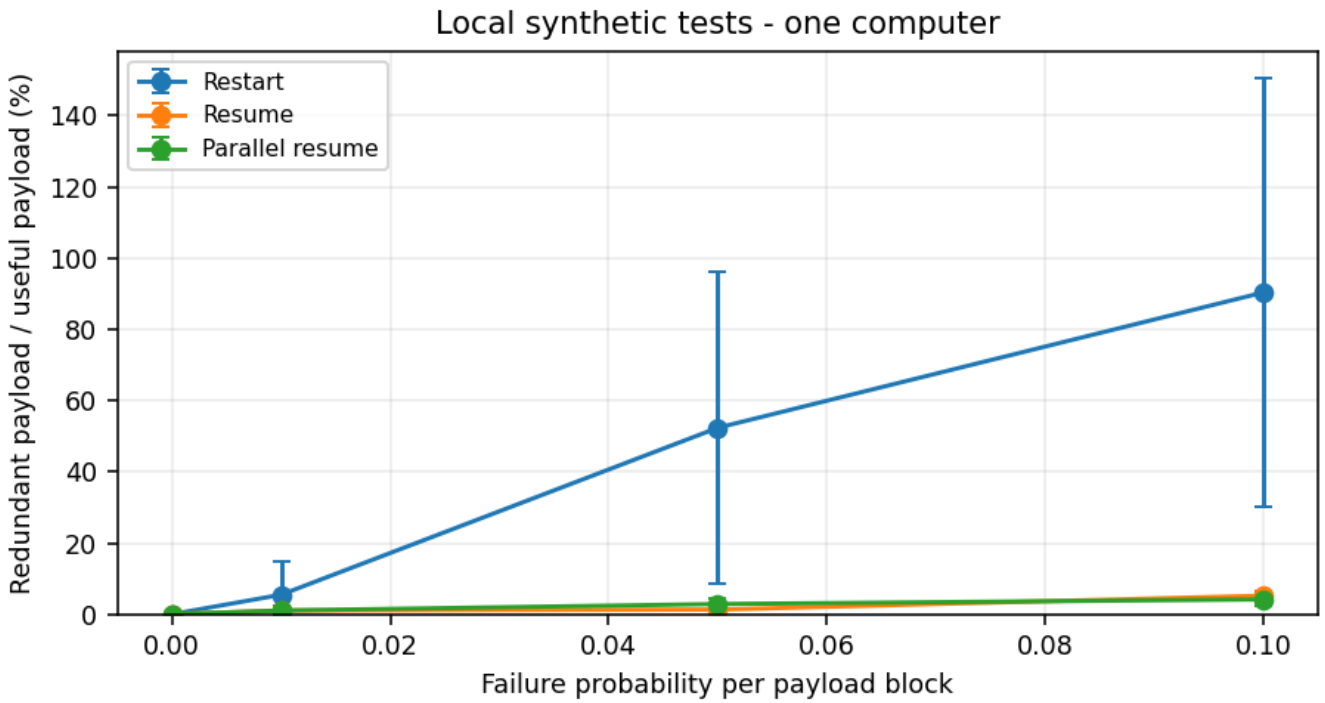
| Part II case | p per block | Runs | Goodput MiB/s | Redundancy % |
|---|---|---|---|---|
| Case 1 | 0 | 3 | 369.57 | 0.00 |
| Case 1 | 0.01 | 3 | 202.39 | 5.47 |
| Case 1 | 0.05 | 3 | 69.39 | 52.34 |
| Case 1 | 0.1 | 3 | 36.95 | 90.36 |
| Case 2 | 0 | 3 | 216.46 | 0.00 |
| Case 2 | 0.01 | 3 | 147.43 | 1.04 |
| Case 2 | 0.05 | 3 | 112.39 | 1.30 |
| Case 2 | 0.1 | 3 | 50.68 | 5.21 |
| Enhanced | 0 | 3 | 307.72 | 0.00 |
| Enhanced | 0.01 | 3 | 232.15 | 1.04 |
| Enhanced | 0.05 | 3 | 153.10 | 2.86 |
| Enhanced | 0.1 | 3 | 126.80 | 4.17 |




Part I: sequential and concurrent aggregate goodput.




Part II: goodput versus failure probability.




Part II: redundant payload divided by useful payload.

## Mean byte counts and completion time

Repeat MiB is redundant payload; Protocol KiB is application framing and acknowledgements. TCP/IP headers and kernel retransmissions are excluded. Values are means over successful repeats.

| Case / mode / n / p | Useful MiB | Sent MiB | Repeat MiB | Time s | Protocol KiB |
|---|---|---|---|---|---|
| Part1 / multi / 1 / 0 | 1.000 | 1.000 | 0.000 | 0.0104 | 0.77 |
| Part1 / multi / 2 / 0 | 2.000 | 2.000 | 0.000 | 0.0089 | 1.54 |
| Part1 / multi / 4 / 0 | 4.000 | 4.000 | 0.000 | 0.0164 | 3.08 |
| Part1 / multi / 8 / 0 | 8.000 | 8.000 | 0.000 | 0.0246 | 6.17 |
| Part1 / single / 1 / 0 | 1.000 | 1.000 | 0.000 | 0.0110 | 0.77 |
| Part1 / single / 2 / 0 | 2.000 | 2.000 | 0.000 | 0.0088 | 1.54 |
| Part1 / single / 4 / 0 | 4.000 | 4.000 | 0.000 | 0.0198 | 3.08 |
| Part1 / single / 8 / 0 | 8.000 | 8.000 | 0.000 | 0.0442 | 6.17 |
| Case1 / multi / 4 / 0 | 4.000 | 4.000 | 0.000 | 0.0108 | 3.08 |
| Case1 / multi / 4 / 0.01 | 4.000 | 4.219 | 0.219 | 0.0221 | 3.29 |
| Case1 / multi / 4 / 0.05 | 4.000 | 6.094 | 2.094 | 0.0604 | 5.38 |
| Case1 / multi / 4 / 0.1 | 4.000 | 7.615 | 3.615 | 0.1203 | 7.94 |
| Case2 / multi / 4 / 0 | 4.000 | 4.000 | 0.000 | 0.0185 | 3.08 |
| Case2 / multi / 4 / 0.01 | 4.000 | 4.042 | 0.042 | 0.0292 | 3.53 |
| Case2 / multi / 4 / 0.05 | 4.000 | 4.052 | 0.052 | 0.0356 | 3.65 |

| Case / mode / n / p | Useful MiB | Sent MiB | Repeat MiB | Time s | Protocol KiB |
|---|---|---|---|---|---|
| Case2 / multi / 4 / 0.1 | 4.000 | 4.208 | 0.208 | 0.0841 | 5.34 |
| Case2 Enh / multi / 4 / 0 | 4.000 | 4.000 | 0.000 | 0.0140 | 4.38 |
| Case2 Enh / multi / 4 / 0.01 | 4.000 | 4.042 | 0.042 | 0.0216 | 4.84 |
| Case2 Enh / multi / 4 / 0.05 | 4.000 | 4.115 | 0.115 | 0.0285 | 5.63 |
| Case2 Enh / multi / 4 / 0.1 | 4.000 | 4.167 | 0.167 | 0.0401 | 6.18 |

## Measured observations

At 1 client(s), multi-threaded mean goodput / single-threaded mean goodput = 1.03.

At 2 client(s), multi-threaded mean goodput / single-threaded mean goodput = 0.99.

At 4 client(s), multi-threaded mean goodput / single-threaded mean goodput = 1.26.

At 8 client(s), multi-threaded mean goodput / single-threaded mean goodput = 1.80.

At p=0, restart redundancy = 0.00%; checkpoint redundancy = 0.00%.

At p=0, Enhanced / Case 2 mean goodput = 1.42; this run favoured Enhanced.

At p=0.01, restart redundancy = 5.47%; checkpoint redundancy = 1.04%.

At p=0.01, Enhanced / Case 2 mean goodput = 1.57; this run favoured Enhanced.

At p=0.05, restart redundancy = 52.34%; checkpoint redundancy = 1.30%.

At p=0.05, Enhanced / Case 2 mean goodput = 1.36; this run favoured Enhanced.

At p=0.1, restart redundancy = 90.36%; checkpoint redundancy = 5.21%.

At p=0.1, Enhanced / Case 2 mean goodput = 2.50; this run favoured Enhanced.

Parallel connections overlap file and acknowledgement waits. They can also add thread, metadata and scheduling overhead. Disk speed, network capacity and unequal file sizes can limit improvement. The observations above describe this measured setup only.

These runs use only 8 generated binary files totalling 1 MiB, over loopback on one computer. Short timing windows, process launches, cache effects and scheduling cause substantial variation. They validate measurement collection and recovery trends; they cannot establish physical LAN throughput.

## 14. Reproduction and submission

```
make -C Part1
Part1/server 9000 /path/to/raw-img single
Part1/client <SERVER_LAN_IP> 9000 cane output_part1

make -C Part2_Case2
Part2_Case2/server 9000 /path/to/raw-img 0.0001
Part2_Case2/client <SERVER_LAN_IP> 9000 cane output_case2
```

Run server and client in separate terminals. Use the same required Part II CLI in Case1, Case2 and Enhanced. Do not run the manual server at the same time as the automated LAN runner. After the physical tests, complete team_members.csv and execute python3 Tools/finalize.py --lan Results/lan. It regenerates the report and graphs, checks required evidence, and produces FinalSubmission_Group_099_107.zip.

## References and preparation

Primary references: the supplied IT305 socket programming assignment, the sample design key, and the submitted Design_Document_Group_099_107_.pdf. Dataset: Alessio Corrado, Animals10, Kaggle, version 2: https://www.kaggle.com/datasets/alessiocorrado99/animals10. WSL networking guidance: Microsoft Learn, https://learn.microsoft.com/en-us/windows/wsl/networking.

ChatGPT assistance was used to revise the code, experiment tools and report. Group members must review the implementation, confirm their actual contributions, perform the lab measurements and follow the course attribution rules. Individual work is not assigned on a member's behalf.